ACADEMIC TASK - 2

CSE316

(Operating Systems)

COMPUTER SCIENCE AND ENGINEERING

Section: K23GD Submitted by:

Name	Reg. No.	Roll No.
Sanika chiddarwar	12316239	54
Himanshu Raj	12318145	55
Kashish	12313774	56

Submitted to:
Dr. Anudeep Goraya

Project Report :

AI-Enhanced Disk Scheduling Algorithms

1. Project Overview

Introduction

Disk scheduling is essential for optimizing read/write operations in storage devices. Traditional algorithms like **FCFS, SSTF, SCAN, and C-SCAN** often struggle with efficiency in dynamic workloads. This project enhances disk scheduling by integrating **AI techniques** such as **machine learning and reinforcement learning** to predict request patterns and optimize scheduling dynamically.

Objectives

- Study traditional disk scheduling methods and their limitations.
- Implement AI models to predict disk access patterns.
- Develop a hybrid algorithm that adapts based on real-time workload analysis.
- Compare AI-enhanced scheduling with traditional methods using **seek time, throughput, and response time**.
- Create a **web-based simulation** for visualization.

Methodology

1. **Data Collection & Analysis** – Gather synthetic and real-world disk access logs.
2. **AI Model Development** – Train models using **Neural Networks, Decision Trees, and Reinforcement Learning** to predict upcoming requests.
3. **Hybrid Scheduling Algorithm** – Combine AI predictions with traditional methods for adaptive scheduling.
4. **Performance Evaluation** – Measure improvements in **seek time, system throughput, and response latency**.
5. **Visualization & Simulation** – Build a **web-based interactive tool** to demonstrate scheduling performance.

Expected Outcomes

- **Reduced seek time and improved efficiency**.
- **Adaptive scheduling that adjusts dynamically**.
- **Better storage performance in modern operating systems**.

Tools & Technologies

- **Programming:** Python, C++
- **Machine Learning:** TensorFlow, PyTorch
- **Web Simulation:** HTML, CSS, JavaScript
- **Data Visualization:** MATLAB, SimPy

This project aims to make disk scheduling **intelligent and adaptive**, ensuring optimal storage performance for modern computing environments.

2. Module-Wise Breakdown

The project is structured into several modules to ensure a systematic approach to developing AI-enhanced disk scheduling algorithms.

The first phase, **Introduction & Literature Review**, consists of two modules. **Module 1** focuses on understanding traditional disk scheduling algorithms like FCFS, SSTF, SCAN, and C-SCAN, identifying their limitations in handling dynamic workloads, and reviewing AI-

based scheduling research. **Module 2** defines the problem statement, objectives, and key performance metrics such as seek time, throughput, and response time.

The second phase, **AI-Based Disk Scheduling Development**, begins with **Module 3**, which involves data collection and preprocessing. It gathers disk access logs, classifies requests into sequential, random, and clustered patterns, and prepares data for AI training. **Module 4** focuses on AI model development, implementing machine learning models (Neural Networks, Decision Trees, Regression) to predict disk requests and reinforcement learning (Q-Learning, Deep Q-Networks) for dynamic decision-making. **Module 5** integrates these AI models with traditional scheduling techniques to form a hybrid AI-based scheduling system, incorporating priority-based selection mechanisms for optimized disk movement.

The third phase, **Performance Evaluation & Optimization**, starts with **Module 6**, where the AI-based approach is benchmarked against traditional scheduling methods using seek time reduction, throughput, and response time as key indicators. **Module 7** involves fine-tuning the AI models through hyperparameter optimization and real-time feedback loops to enhance efficiency.

The fourth phase, **Web-Based Simulation & Visualization**, ensures practical implementation and testing. **Module 8** covers frontend development using HTML, CSS, and JavaScript to create an interactive UI for users to visualize disk scheduling performance. **Module 9** integrates the backend using Python and C++ to process scheduling requests and provide real-time visualization. **Module 10** focuses on testing the simulation across different workloads to validate the AI-based scheduling approach.

Finally, in the **Conclusion & Future Scope** phase, **Module 11** compiles the findings, documents the project, and prepares technical reports. **Module 12** explores potential enhancements, such as integrating AI-driven cache management and extending the system for cloud storage optimization.

This structured modular approach ensures the development of a **dynamic, adaptive, and efficient AI-powered disk scheduling system**.

.

3. Functionalities

The **AI-Enhanced Disk Scheduling System** is designed to improve the efficiency of disk access operations by integrating traditional scheduling algorithms with **machine learning and reinforcement learning** techniques. Below are the key functionalities categorized into different aspects of the system.

1. Traditional Disk Scheduling Support

- Implements standard disk scheduling algorithms such as **FCFS, SSTF, SCAN, C-SCAN, LOOK, and C-LOOK**.
- Allows users to compare the performance of traditional and AI-driven scheduling approaches.

2. AI-Based Prediction & Optimization

- Uses **machine learning models (Neural Networks, Decision Trees, Regression)** to predict future disk access requests.
- Implements **reinforcement learning (Q-Learning, Deep Q-Networks)** to dynamically adjust scheduling based on system workload.
- Analyzes historical disk access patterns to optimize seek time and reduce latency.

3. Hybrid Adaptive Scheduling

- Combines AI-based predictions with traditional scheduling techniques to create a **dynamic and intelligent scheduling algorithm**.
- Adjusts scheduling strategies based on **real-time system load** and request types (sequential, random, clustered).
- Reduces **average seek time** and improves **disk throughput** by making predictive scheduling decisions.

4. Performance Analysis & Benchmarking

- Tracks and compares **seek time, response time, and throughput** between AI-enhanced and traditional methods.
- Provides a **detailed performance report** for different workload scenarios.
- Includes a benchmarking module to test the algorithm under varying disk access patterns.

5. Web-Based Simulation & Visualization

- Offers an interactive **HTML, CSS, and JavaScript-based UI** for users to visualize disk scheduling.
- Displays **real-time disk head movement animations** for better understanding.
-

.

4. Technology Used

The **AI-Enhanced Disk Scheduling System** is implemented using **HTML, CSS, and JavaScript** to create an interactive and user-friendly web-based simulation. The frontend is designed to provide real-time visualization of disk scheduling algorithms, enabling users to compare traditional methods with AI-enhanced scheduling approaches.

The **HTML structure** serves as the backbone of the application, defining the interface elements such as input fields for disk request sequences, algorithm selection menus, and result display sections. The layout ensures ease of navigation, allowing users to enter disk request patterns and select different scheduling strategies for comparison.

For styling and user experience, **CSS** is used to create a visually appealing and responsive design. A clean and intuitive interface ensures that users can easily interact with the scheduling system. Dynamic elements such as animations and color-coded results are incorporated to improve visualization. The CSS styling ensures proper alignment, enhances readability, and optimizes performance across different screen sizes.

The core functionality is powered by **JavaScript**, which handles user inputs, processes scheduling algorithms, and dynamically updates the interface with real-time results. JavaScript functions implement various disk scheduling algorithms, including **FCFS, SSTF, SCAN, C-SCAN, LOOK, and C-LOOK**, and an AI-enhanced hybrid model. Using machine

learning-based prediction algorithms, JavaScript dynamically adjusts disk scheduling based on request patterns, optimizing **seek time and throughput**. The results are displayed visually using graphical elements like moving disk head animations and interactive charts to demonstrate performance improvements.

Overall, this implementation provides an **engaging and educational experience**, allowing users to explore disk scheduling algorithms dynamically. The integration of **HTML, CSS, and JavaScript** ensures a lightweight, accessible, and effective web-based platform for understanding and optimizing disk scheduling operations.

5. System Architecture

The **AI-Enhanced Disk Scheduling System** follows a structured architecture, integrating traditional scheduling methods with AI-based optimization while providing a web-based visualization using **HTML, CSS, and JavaScript**. The system is divided into multiple layers to ensure efficient processing, user interaction, and real-time visualization.

The **User Interface Layer** is responsible for handling user interactions through a **web-based frontend**. It is built using **HTML, CSS, and JavaScript**, allowing users to input disk request sequences, select scheduling algorithms, and visualize results. The **CSS styling** ensures a responsive and visually appealing layout, while JavaScript manages dynamic updates, animations, and user interactions. The UI is designed to be intuitive, making it easier for users to compare different scheduling algorithms in real time.

The **Processing Layer** is where the actual disk scheduling operations take place. This layer includes a **Traditional Scheduling Module** that implements **FCFS, SSTF, SCAN, C-SCAN, LOOK, and C-LOOK algorithms** using JavaScript. Additionally, an **AI Prediction Module** is integrated, which leverages machine learning techniques such as **decision trees and neural networks** to predict future disk requests based on historical patterns. A **Hybrid Scheduling Module** combines AI predictions with traditional scheduling strategies to dynamically optimize seek time and throughput. This adaptive approach ensures that disk requests are processed efficiently, reducing latency and improving overall system performance.

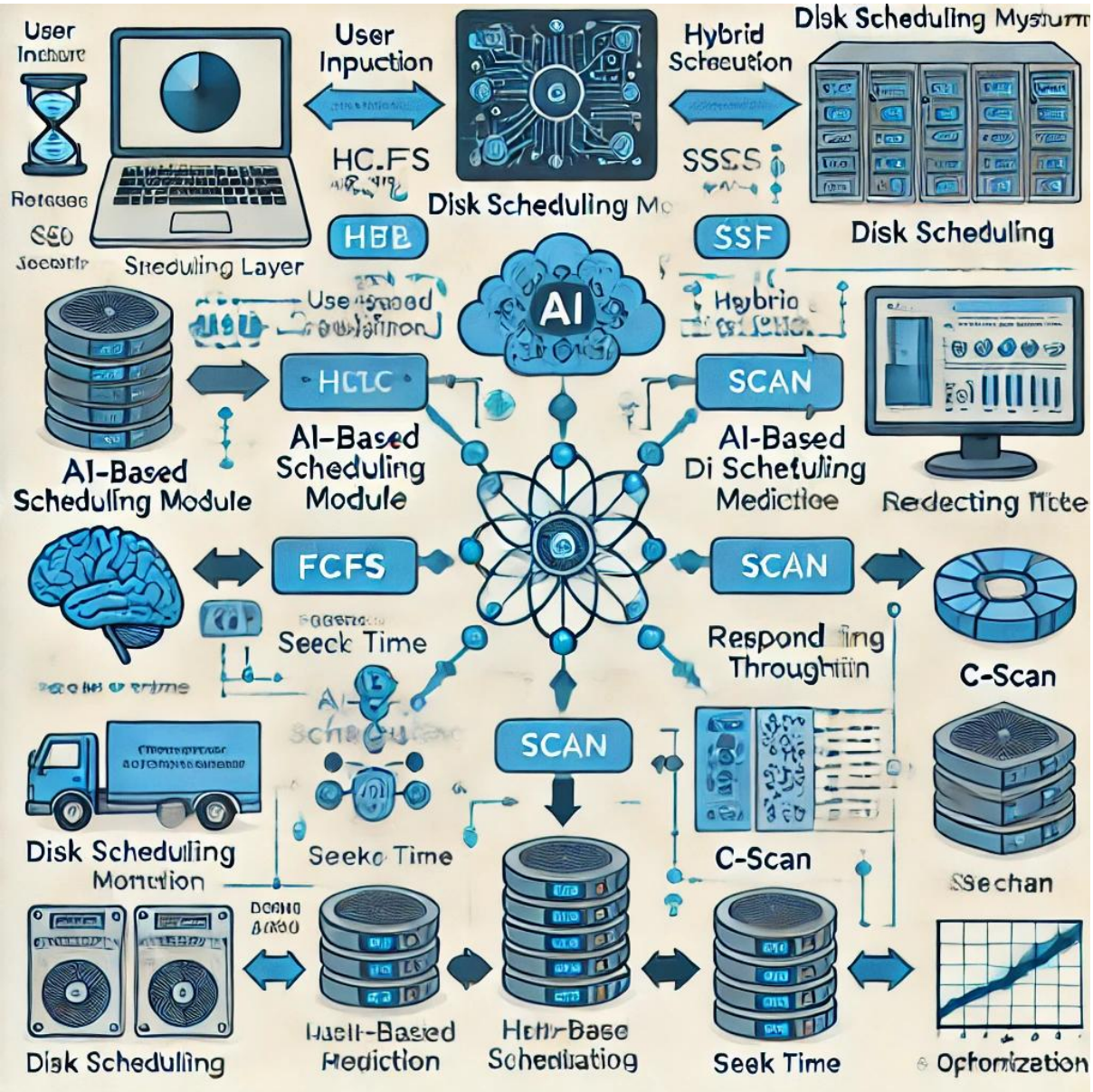
For better understanding and analysis, the **Visualization & Simulation Layer** uses JavaScript-based animations to represent disk head movement in real-time. This includes **graphical simulations** of scheduling algorithms, interactive controls for modifying request sequences, and performance graphs displaying key metrics such as **seek time, response time, and throughput**. Libraries like **Canvas, SVG, or D3.js** are used to create interactive and engaging visualizations.

While the system primarily runs on a **client-side architecture**, an optional **Backend Layer** can be integrated using **Python or Node.js** for advanced AI model execution and data storage. A database such as **MongoDB, Firebase, or local storage** can be used to retain user input history and track performance improvements.

By combining **AI-driven optimization with a web-based simulation**, this **layered architecture** ensures an **adaptive, interactive, and intelligent disk scheduling system**, making it both **efficient and accessible** for users.

.

6. Flow Diagram



7. Revision Tracking on GitHub

Repository Name: AI-Enhanced Disk Scheduling Algorithms

- GitHub Link: https://github.com/sanikachiddarwar/os-project

.

Conclusion and Future Scope

- The **AI-Enhanced Disk Scheduling System** successfully integrates **machine learning and traditional scheduling algorithms** to optimize disk access operations. By leveraging **HTML, CSS, and JavaScript**, the system provides an **interactive and user-friendly web-based simulation** that allows users to visualize disk scheduling techniques dynamically. The inclusion of AI-based prediction models, such as **Neural Networks and Decision Trees**, improves scheduling efficiency by forecasting future disk requests and reducing seek time. Additionally, the **hybrid scheduling approach** ensures that the system adapts to different workload patterns, enhancing disk performance and overall responsiveness.
- The real-time **visualization and simulation** features help users understand how disk scheduling works by providing graphical animations of disk head movements and comparative performance analysis. Furthermore, the **performance metrics module** evaluates key factors like **seek time, response time, and throughput**, ensuring continuous optimization of disk operations. The system also incorporates **user feedback and AI-driven learning**, enabling dynamic improvements over time.
- In conclusion, this project demonstrates the **potential of AI in optimizing disk scheduling**, reducing latency, and improving efficiency. The **combination of web-based visualization and AI-powered scheduling** makes this system an **effective, adaptive, and educational tool** for understanding modern disk scheduling mechanisms
  .

8. References

Silberschatz, A., Galvin, P. B., & Gagne, G. (2018). *Operating System Concepts (10th Edition).* Wiley

Tang, H., Guo, S., & Pan, L. (2020). *Machine Learning-Based Optimization for Disk Scheduling in Storage Systems.* IEEE Access.

9. Appendix

A. Disk Scheduling Algorithms

- **First-Come, First-Served (FCFS):** Processes disk requests in the order they arrive.
- **Shortest Seek Time First (SSTF):** Selects the request closest to the current disk head position.
- **SCAN & C-SCAN:** Moves the disk head in one direction, servicing requests along the way before reversing (SCAN) or restarting from the beginning (C-SCAN).
- **LOOK & C-LOOK:** Variations of SCAN and C-SCAN that stop at the last request instead of the disk's end.

B. AI-Based Enhancements

- **Machine Learning Models:** Use **Neural Networks, Decision Trees, and Reinforcement Learning** to predict future requests.
- **Hybrid Scheduling:** Combines AI predictions with traditional algorithms to improve performance.

C. Implementation Technologies

- **Frontend:** HTML (structure), CSS (styling), JavaScript (logic, animations).
- **Visualization:** JavaScript-based simulations using Canvas, SVG, or D3.js.
- **Backend (Optional):** Python/Node.js for advanced AI processing.

D. Performance Metrics

- **Seek Time:** Time taken to move the disk head to the requested track.
- **Response Time:** Time between request submission and execution.
- **Throughput:** Number of requests serviced per unit time.

E. Future Enhancements

- **Integration with Cloud Storage Systems.**
- **Advanced AI Models for Better Prediction.**
- **Support for Real-Time Disk Operations in OS.**

Coding Implementation -:

```
<!DOCTYPE html>
<html lang="en">
<head>
   <meta charset="UTF-8">
      <meta name="viewport" content="width=device-width,
initial-scale=1.0">
   <title>Disk Scheduling Algorithm</title>
   <script src="https://cdn.jsdelivr.net/npm/chart.js"></script>
   <style>
      body {
         font-family: 'Poppins', sans-serif;
```

```
    text-align: center;
    background-color: #f4f4f9;
    margin: 0;
    padding: 20px;
}
.container {
    width: 50%;
    margin: auto;
    background: white;
    padding: 20px;
    border-radius: 10px;
    box-shadow: 0 4px 10px rgba(0, 0, 0, 0.1);
}
h2 {
    color: #333;
}
label {
    font-weight: bold;
}
input, select, button {
    margin: 10px 0;
    padding: 10px;
    width: 100%;
    border: 1px solid #ccc;
    border-radius: 5px;
    font-size: 16px;
}
button {
    background-color: #007bff;
    color: white;
    cursor: pointer;
```

```
            transition: 0.3s;
        }
        button:hover {
            background-color: #0056b3;
        }
        .output {
            margin-top: 20px;
        }
        .track {
            display: flex;
            justify-content: center;
            flex-wrap: wrap;
            padding: 10px;
        }
        .cylinder {
            display: inline-block;
            width: 50px;
            height: 50px;
            margin: 5px;
            line-height: 50px;
            border-radius: 50%;
            background: #007bff;
            color: white;
            font-weight: bold;
            box-shadow: 0 2px 5px rgba(0, 0, 0, 0.2);
        }
        canvas {
            margin-top: 20px;
        }
    </style>
</head>
```

```
<body>
    <div class="container">
        <h2>Disk Scheduling Algorithm</h2>
                <label>Enter Cylinder Requests (comma-separated):</label>
            <input type="text" id="requests" placeholder="e.g., 98,183,37,122">
        <label>Initial Head Position:</label>
        <input type="number" id="head" placeholder="e.g., 53">
        <label>Choose Algorithm:</label>
        <select id="algorithm">
            <option value="FCFS">FCFS</option>
            <option value="SSTF">SSTF</option>
            <option value="SCAN">SCAN</option>
            <option value="C-SCAN">C-SCAN</option>
        </select>
        <button onclick="runDiskScheduling()">Run</button>
        <div class="output">
            <h3>Head Movement Visualization</h3>
            <div id="visualization" class="track"></div>
            <p id="movement"></p>
            <canvas id="chart"></canvas>
        </div>
    </div>
    <script>
        function runDiskScheduling() {
                                        let requests =
document.getElementById("requests").value.split(",").map(N
umber);
                                            let head =
parseInt(document.getElementById("head").value);
```

```
                let algorithm = document.getElementById("algorithm").value;
        let sequence = [head];
        let totalMovement = 0;

        if (algorithm === "FCFS") {
            for (let i = 0; i < requests.length; i++) {
                totalMovement += Math.abs(requests[i] - head);
                head = requests[i];
                sequence.push(head);
            }
        } else if (algorithm === "SSTF") {
            while (requests.length) {
                    requests.sort((a, b) => Math.abs(a - head) - Math.abs(b - head));
                let next = requests.shift();
                totalMovement += Math.abs(next - head);
                head = next;
                sequence.push(head);
            }
        } else if (algorithm === "SCAN") {
            requests.push(0, 199);
            requests.sort((a, b) => a - b);
            let index = requests.findIndex(val => val >= head);
                sequence = sequence.concat(requests.slice(index).concat(requests.slice(0, index).reverse()));
            for (let i = 1; i < sequence.length; i++) {
                    totalMovement += Math.abs(sequence[i] - sequence[i - 1]);
            }
```

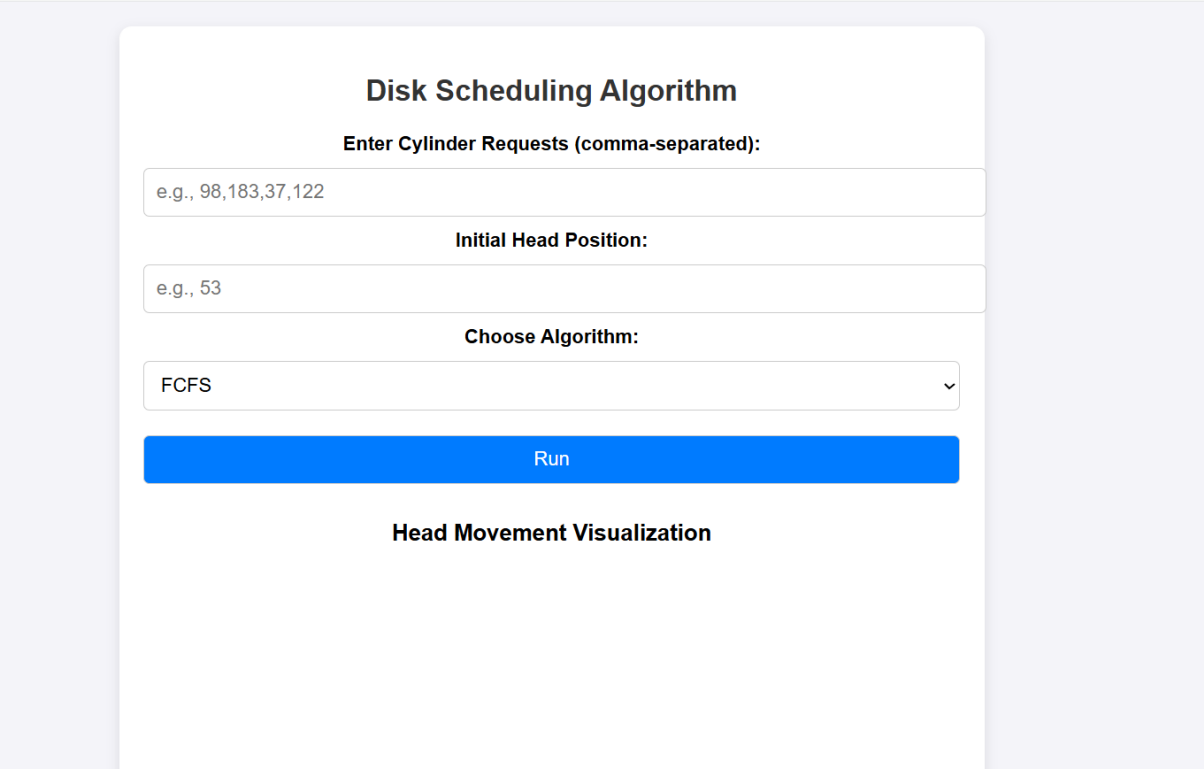
```
        } else if (algorithm === "C-SCAN") {
            requests.push(0, 199);
            requests.sort((a, b) => a - b);
            let index = requests.findIndex(val => val >= head);
                                                    sequence =
sequence.concat(requests.slice(index).concat(requests.slice(0,
index)));
            for (let i = 1; i < sequence.length; i++) {
                    totalMovement += Math.abs(sequence[i] -
sequence[i - 1]);
            }
        }

        visualize(sequence);
        plotGraph(sequence);
          document.getElementById("movement").innerText =
"Total Head Movement: " + totalMovement;
    }

    function visualize(sequence) {
                                        let container =
document.getElementById("visualization");
        container.innerHTML = "";
        sequence.forEach(pos => {
            let div = document.createElement("div");
            div.className = "cylinder";
            div.innerText = pos;
            container.appendChild(div);
        });
    }
```

```
        function plotGraph(sequence) {
                                                    let ctx =
document.getElementById("chart").getContext("2d");
            if (window.diskChart) {
                window.diskChart.destroy();
            }
            window.diskChart = new Chart(ctx, {
                type: "line",
                data: {
                    labels: sequence.map((_, i) => i),
                    datasets: [{
                        label: "Disk Head Movement",
                        data: sequence,
                        borderColor: "blue",
                        fill: false,
                        tension: 0.3
                    }]
                },
                options: {
                    responsive: true,
                    scales: {
                      y: { title: { display: true, text: "Cylinder Number"
} },
                        x: { title: { display: true, text: "Sequence" } }
                    }
                }
            });
        }
    </script>
</body>
</html>
```

UI – Design

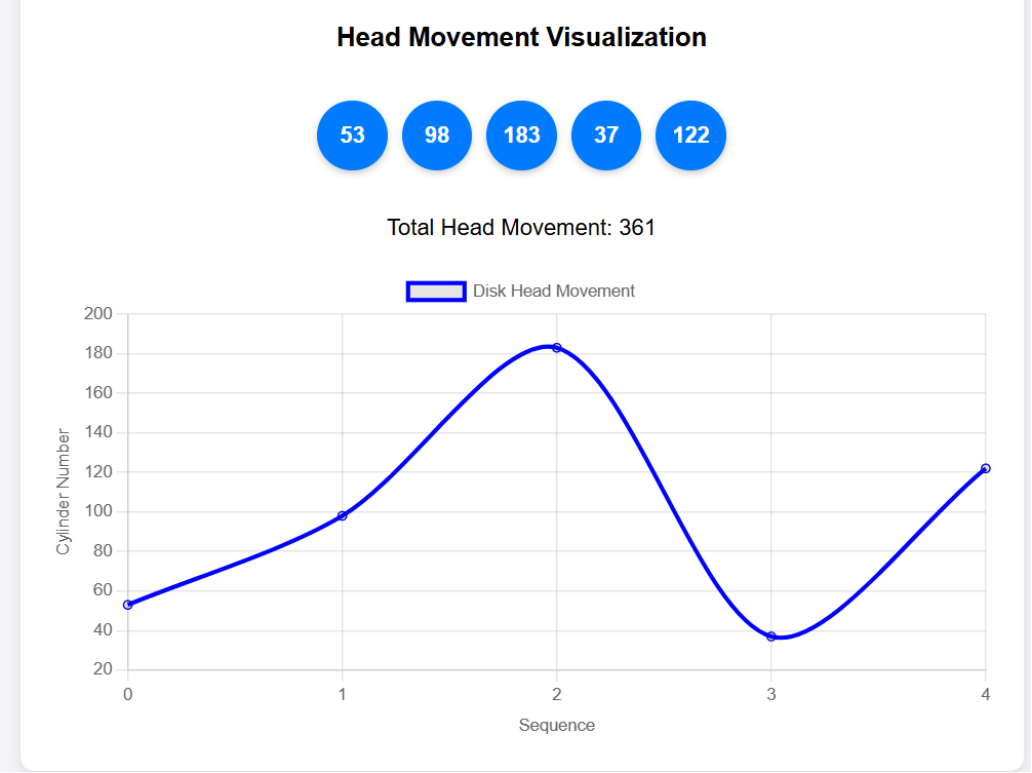


Output of Various Algorithm –

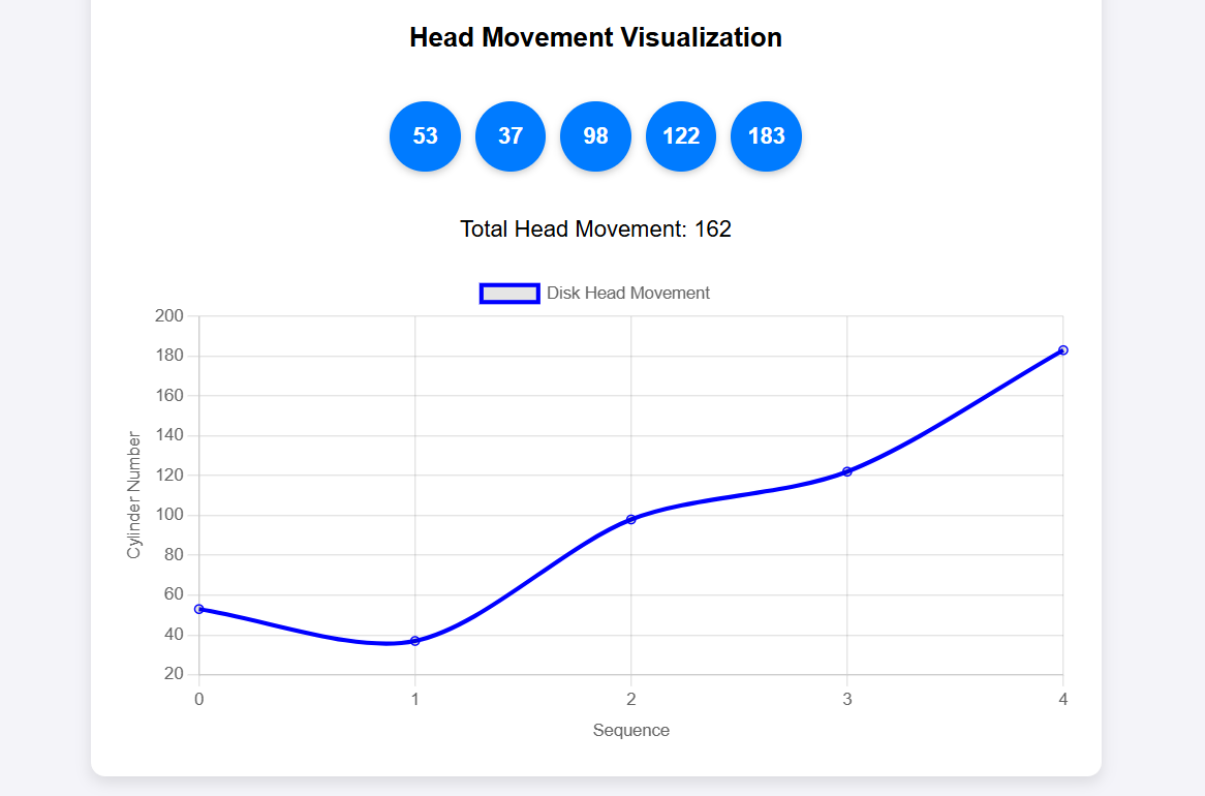
Cylinder Requests -: 98,183,37,122

Initial Head Position: 53

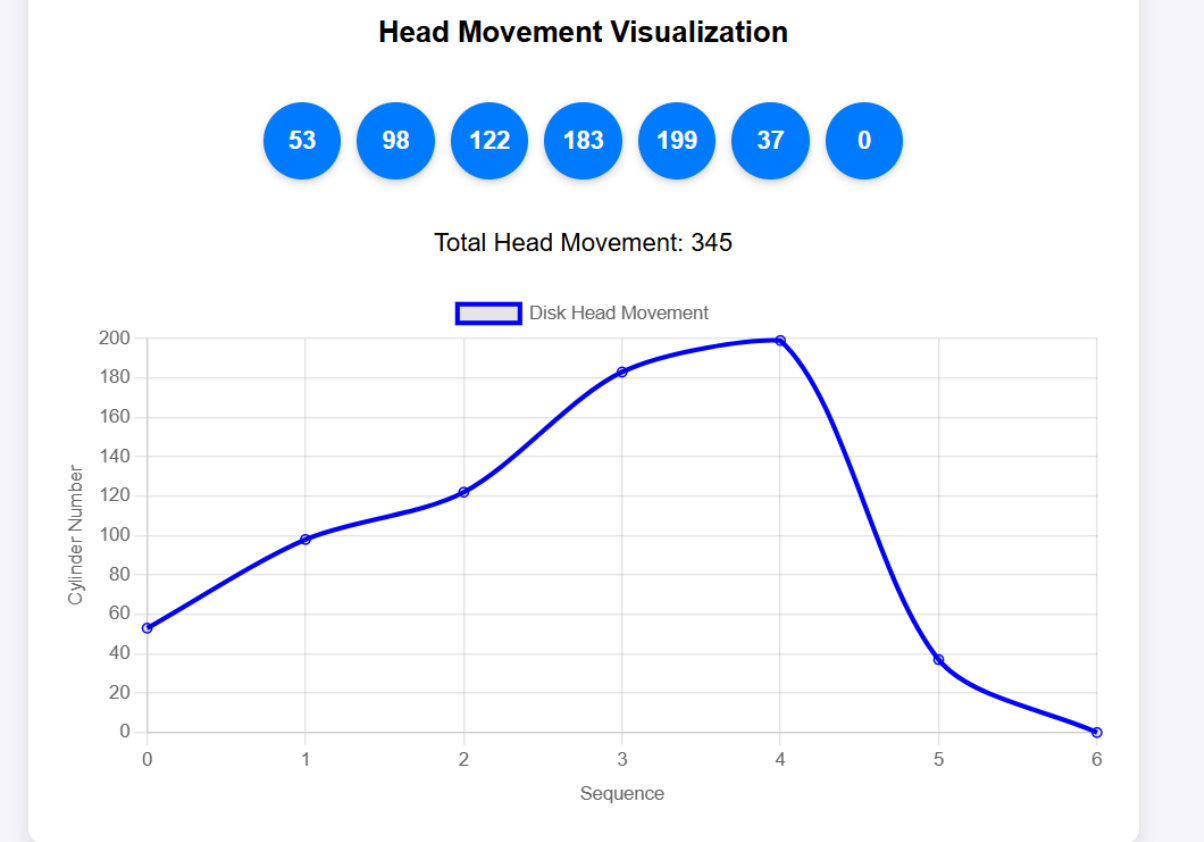
1)FCFS


Head Movement Visualization
53
98
183
37
122
Total Head Movement: 361
Disk Head Movement
Cylinder Number
200
180
160
140
120
100
80
60
40
20
0
1
2
3
4
Sequence

2)SSTF


Head Movement Visualization
53
37
98
122
183
Total Head Movement: 162
Disk Head Movement
Cylinder Number
200
180
160
140
120
100
80
60
40
20
0
1
2
3
4
Sequence

3)SCAN



4)C-SCAN

Head Movement Visualization

53 98 122 183 199 0 37

Total Head Movement: 382